MÓDULO 4:

Aprendizaje por Refuerzo

INTELIGENCIA ARTIFICIAL APLICADA A LA EMPRESA

Tabla de Contenido

Introducción 3

Objetivos 4

Tema 1. Introducción al aprendizaje por refuerzo 5

1.1 Agente, entorno y recompensas 6

1.2 Política, función de valor y función Q 17

Tema 2. Algoritmos comunes 25

2.1 Q-learning 25

2.2 SARSA 31

2.3 Aplicaciones reales (juegos, robótica) 38

Resumen 46

Introducción

El **aprendizaje por refuerzo** es una rama fundamental dentro del campo del aprendizaje automático que se basa en la idea de que un agente puede aprender a tomar decisiones óptimas a través de la interacción con un entorno.

A diferencia de otros enfoques, aquí el conocimiento no se transmite directamente, sino que se construye mediante la experiencia, el **ensayo-error** y la retroalimentación en forma de recompensas o penalizaciones.

Este paradigma se inspira en procesos cognitivos y conductuales observados en seres vivos, lo que lo convierte en una herramienta poderosa para el desarrollo de sistemas autónomos inteligentes.

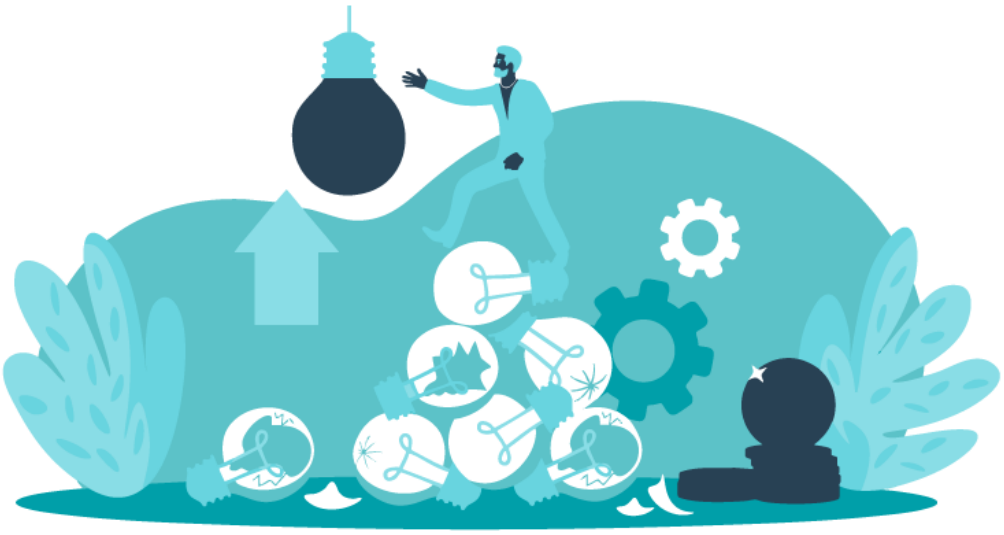
Este módulo introduce al estudiante en los conceptos esenciales de esta disciplina, presentando elementos clave como los **agentes**, los **entornos**, las **recompensas**, las **políticas** de decisión, y funciones como la **función de valor** y la **función Q**. A partir de estos cimientos, se exploran algoritmos fundamentales como **Q-learning** y **SARSA**, explicando su funcionamiento, fortalezas y diferencias.

La importancia de este tema radica en su aplicación en áreas de vanguardia como la **robótica**, el **desarrollo de videojuegos** o los **sistemas de recomendación adaptativos**. Comprender sus principios permite a los futuros profesionales diseñar sistemas capaces de aprender de forma autónoma en contextos complejos y cambiantes, lo cual aporta un valor añadido significativo en entornos empresariales innovadores.

Objetivos

A continuación, se presentan los objetivos que persigue el desarrollo de este módulo:

- **Comprender** los conceptos fundamentales del aprendizaje por refuerzo y su diferencia con otros tipos de aprendizaje automático.
- **Conocer** los componentes básicos del proceso de aprendizaje por refuerzo: agente, entorno, recompensas, política y funciones de valor.
- **Aprender a** identificar y aplicar los algoritmos más representativos como Q-learning y SARSA en contextos prácticos.
- **Definir** aplicaciones reales del aprendizaje por refuerzo en sectores como la robótica, los videojuegos y otros entornos dinámicos.



Tema 1. Introducción al aprendizaje por refuerzo

El aprendizaje por refuerzo representa una de las formas más interesantes en que un sistema puede aprender a tomar decisiones a partir de la experiencia.

En lugar de seguir ejemplos directos o patrones predefinidos, este enfoque permite que un agente **descubra progresivamente** cuáles son las acciones más eficaces en un entorno mediante la retroalimentación obtenida por recompensas o castigos.

Esta metodología resulta especialmente útil en situaciones donde no se cuenta con información completa o directa sobre cuál es la mejor estrategia a seguir.

Su utilidad se extiende a campos como la automatización, la robótica, los videojuegos y los sistemas inteligentes, donde el entorno cambia constantemente y el sistema debe adaptarse.

Este tipo de aprendizaje permite desarrollar **soluciones flexibles y autónomas**, capaces de mejorar con la práctica sin intervención humana directa. Conocer sus fundamentos y aplicaciones ofrece una visión más amplia de cómo la inteligencia artificial puede abordar problemas complejos de toma de decisiones.

Figura 1: Aprendizaje por refuerzo

1.1 Agente, entorno y recompensas

En el aprendizaje por refuerzo, uno de los elementos más determinantes para comprender su dinámica es el conjunto formado por el **agente**, el **entorno** y el sistema de **recompensas**.

Estos tres pilares interactúan en un ciclo continuo que permite al sistema adquirir experiencia y mejorar su rendimiento progresivamente. A través de la interacción entre estos componentes, se construyen políticas de acción más eficaces, que permiten alcanzar objetivos definidos y resolver tareas complejas en contextos diversos.

1.1.1 El agente: núcleo de la toma de decisiones

El agente es el componente encargado de ejecutar acciones con la intención de maximizar un beneficio acumulado. Representa la parte activa del sistema, responsable de **aprender a través de la experiencia**.

El agente observa el estado actual del entorno, toma una decisión (acción), y recibe como respuesta una recompensa junto con un nuevo estado. A través de este ciclo, va modificando su política para tomar decisiones más acertadas en el futuro.

Este comportamiento puede observarse en múltiples dominios.

Ejemplos:

En el ámbito de los videojuegos, un personaje controlado por IA aprende a esquivar enemigos, recolectar objetos y alcanzar metas con mayor eficacia con cada partida.

En un contexto logístico, un software puede actuar como agente que determina rutas de transporte más eficientes.

Y en el sector bancario, algoritmos de trading automático deciden cuándo entrar o salir de una posición de mercado, analizando las fluctuaciones y ajustando sus estrategias con base en las recompensas obtenidas.

La implementación del agente no está sujeta a una única técnica. Puede estar basado en modelos tabulares simples, como en Q-learning, o en redes neuronales profundas, como en el aprendizaje profundo por refuerzo. La

elección del tipo de agente depende del tamaño del espacio de estados, de la complejidad del entorno y de la necesidad de generalización.

Uno de los retos que enfrenta el agente es cómo equilibrar la **exploración** y la **explotación**. La primera implica probar nuevas acciones que pueden resultar en mejores recompensas a largo plazo, mientras que la segunda se basa en repetir aquellas decisiones que ya han demostrado ser útiles.

Este dilema, conocido como el "**trade-off exploración-explotación**", es un tema recurrente en los sistemas de aprendizaje adaptativo.

1.1.2 El entorno: marco dinámico de interacción

El entorno es el contexto en el cual opera el agente. Proporciona los estados, responde a las acciones del agente y genera las recompensas asociadas. Esta estructura puede representar un espacio físico, como una planta industrial o un vehículo en circulación, o un espacio digital, como un sitio web o una aplicación móvil.

El entorno puede ser determinista o estocástico. En un entorno **determinista**, una misma acción desde un mismo estado siempre produce el mismo resultado. En cambio, en uno **estocástico**, una acción puede tener resultados distintos, aunque se repita en las mismas condiciones.

Esta variabilidad añade complejidad al aprendizaje, ya que obliga al agente a desarrollar políticas más sólidas que se adapten a la incertidumbre.

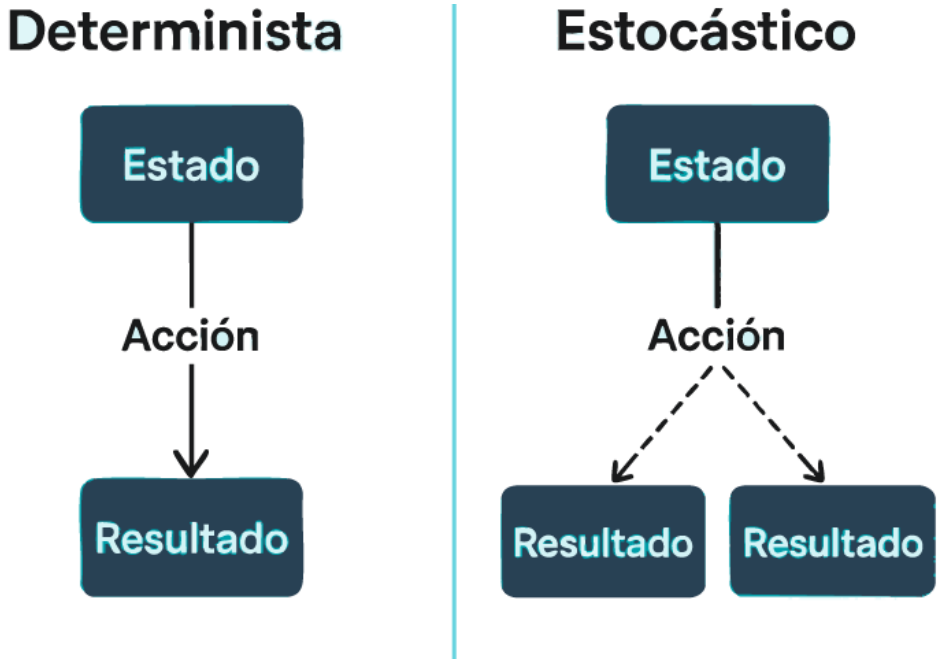


Figura 2: Diferencia entre entorno determinista y estocástico

Ejemplos:

Por ejemplo, un **sistema de atención al cliente** automatizado puede considerar como entorno el conjunto de mensajes del usuario, sus respuestas previas y el historial de conversaciones. En este entorno, el agente propone respuestas y recibe retroalimentación implícita (continuación de la conversación) o explícita (valoraciones del usuario).

Otro caso práctico es el de una **red de distribución energética**, donde el entorno incluye el consumo estimado, la producción de energía renovable y la demanda en tiempo real. Cada acción del agente afecta directamente al equilibrio del sistema.

Para facilitar el entrenamiento y prueba de agentes, se recurre a entornos simulados, como OpenAI Gym o Unity ML-Agents, que permiten replicar contextos físicos o digitales sin los costes asociados a la experimentación en entornos reales.

Estos simuladores han sido decisivos para democratizar el acceso al aprendizaje por refuerzo, permitiendo ensayos más seguros y rápidos.

1.1.3 La recompensa: instrumento de retroalimentación

La **recompensa** es el mecanismo mediante el cual el agente recibe información sobre la calidad de sus decisiones. Generalmente, se expresa como un número que puede ser **positivo, negativo o neutro**, y que permite al agente cuantificar su progreso hacia el objetivo deseado. El diseño de esta señal tiene un impacto decisivo sobre el comportamiento aprendido.

Una recompensa adecuada debe ser alineada con los objetivos del sistema.

Ejemplo:

En **sistemas de gestión de flotas** se puede definir una recompensa positiva por reducir los tiempos de entrega o los costes de combustible. En cambio, se penaliza el incumplimiento de entregas o la sobrecarga de rutas. En estos contextos, la función de recompensa actúa como brújula para que el agente oriente sus decisiones hacia la mejora de resultados.

¿Sabías que?

En el desarrollo de sistemas de IA para videojuegos como StarCraft II, se utilizan recompensas diferidas, que solo se entregan tras una larga serie de acciones.

Esta estructura complica el aprendizaje porque el agente debe atribuir el mérito del éxito a acciones tomadas mucho tiempo atrás. Sin embargo, permite desarrollar comportamientos altamente estratégicos.

El momento en que se otorga la recompensa también influye en el tipo de aprendizaje. Una señal inmediata favorece la reactividad del sistema, mientras que una señal retardada requiere mecanismos de memoria o estimación de largo plazo.

Técnicas como el **aprendizaje temporal diferencial** (Temporal Difference Learning) permiten gestionar estos escenarios, combinando elementos del aprendizaje supervisado con métodos secuenciales.

1.1.4 Ciclo de aprendizaje: dinámica entre los componentes

El ciclo que une al agente, al entorno y a las recompensas se denomina interacción.

A través de este proceso, el agente ejecuta acciones, recibe retroalimentación y ajusta su política. La eficiencia del aprendizaje depende de cómo se modela esta interacción y de los algoritmos utilizados para actualizar la política.

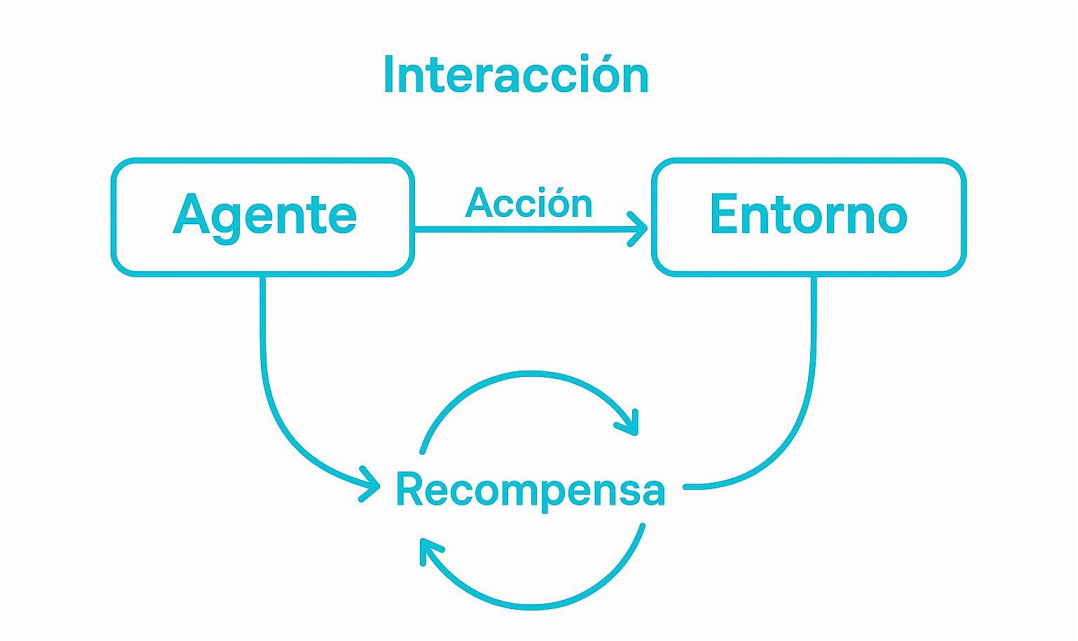


Figura 3: La interacción y el ciclo de aprendizaje

Ejemplos:

Un caso práctico interesante lo encontramos en la industria del transporte urbano.

Un sistema de inteligencia artificial que gestiona flotas de autobuses puede usar aprendizaje por refuerzo para optimizar los horarios, reducir el tiempo de espera y mejorar la experiencia del usuario.

El agente actúa sobre decisiones de asignación de vehículos; el entorno refleja el flujo de pasajeros, el tráfico y la climatología; y las recompensas se basan en métricas como puntualidad y satisfacción del usuario.

La siguiente tabla resume los elementos que conforman el sistema y muestra cómo se relacionan en diferentes aplicaciones:

Elemento	Descripción	Ejemplo aplicado
Agente	Sistema que toma decisiones en función del estado del entorno	Robot logístico, modelo de inversión automatizado
Entorno	Contexto donde se ejecutan las acciones y que proporciona información	Simulación de tráfico urbano, interfaz de usuario
Recompensa	Valor que indica la utilidad de la acción realizada	Puntos por eficiencia energética, feedback de cliente

Tabla 1: Elementos del aprendizaje por refuerzo y ejemplos de aplicación

1.1.5 Estrategias de modelado y buenas prácticas

La definición correcta de los tres elementos básicos del aprendizaje por refuerzo es indispensable para garantizar resultados coherentes. Uno de los errores más comunes es **diseñar funciones de recompensa que incentivan comportamientos no deseados**.

Ejemplos:

En un sistema que premia la rapidez, un agente podría aprender a omitir pasos importantes para obtener una puntuación más alta, pero generando errores o pérdidas.

Para evitar estas desviaciones, se recomienda **validar las recompensas** en simulaciones previas, ajustarlas con pruebas iterativas y evaluar los comportamientos aprendidos en situaciones de estrés. Este tipo de evaluaciones permiten identificar atajos indeseados o estrategias oportunistas que deben ser corregidas.

En cuanto al entorno, es necesario **identificar todos los estados relevantes**. Un modelo demasiado simplificado puede dejar fuera información crítica, mientras que uno demasiado complejo puede ralentizar el aprendizaje o generar ruido. Las herramientas de reducción de dimensionalidad, como PCA, pueden ayudar a representar el entorno de manera más eficiente.

¿Sabías que?

Netflix experimenta con algoritmos de aprendizaje por refuerzo para personalizar las portadas que se muestran a los usuarios. Según sus patrones de consumo, el sistema adapta la imagen de la serie o película para maximizar el número de clics y visualizaciones.

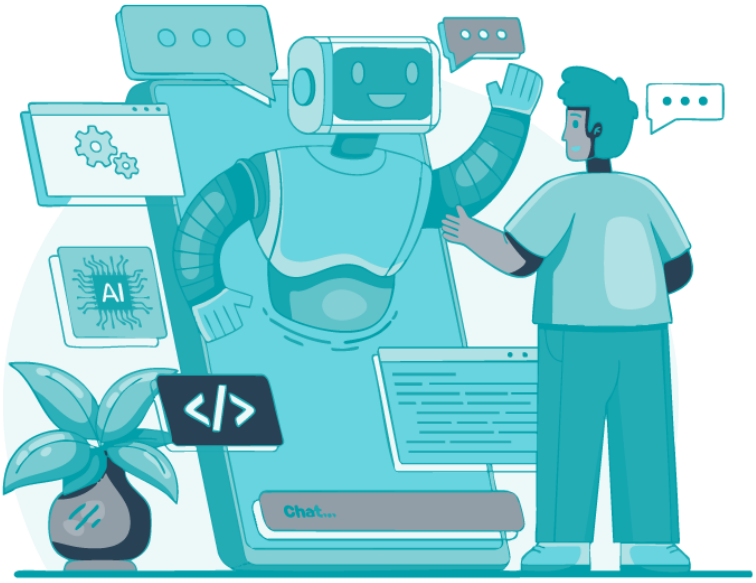
Este tipo de personalización demuestra cómo un diseño cuidadoso del agente y del entorno, unido a una recompensa bien definida, puede mejorar la experiencia del usuario y aumentar los objetivos comerciales.

1.1.6 Avances tecnológicos y perspectivas futuras

El avance en la capacidad de procesamiento y el acceso a bibliotecas especializadas ha facilitado la implementación de modelos complejos de aprendizaje por refuerzo.

Herramientas como TensorFlow Agents, PyTorch RL o Ray RLlib han permitido experimentar con múltiples arquitecturas, desde agentes tabulares hasta políticas basadas en redes convolucionales o recurrentes.

Estas herramientas han propiciado el desarrollo de sistemas más sofisticados, como los vehículos autónomos de Waymo, los bots conversacionales adaptativos, o las plataformas de inversión inteligente.



A medida que estas tecnologías evolucionan, también lo hace el diseño de agentes más robustos, capaces de adaptarse a entornos no estacionarios, dinámicos y multiagente.

Una de las líneas de investigación más prometedoras es la integración del aprendizaje por refuerzo con técnicas de **aprendizaje profundo y modelos generativos.** Esto permitirá que los agentes **anticipen consecuencias**, **simulen escenarios** alternativos y aprendan de manera más eficiente con menos interacción directa.

Así, el aprendizaje basado en simulación se consolidará como una herramienta de gran utilidad en contextos con restricciones operativas o riesgos elevados.

1.2 Política, función de valor y función Q

La comprensión del aprendizaje por refuerzo requiere adentrarse en tres conceptos que estructuran la toma de decisiones del agente: la **política**, la **función de valor** y la **función Q**.

Estos componentes, interconectados pero distintos en su función, conforman el esqueleto que permite al agente seleccionar acciones de forma inteligente en entornos complejos y con información parcial. Estudiarlas en detalle facilita implementar estrategias más robustas y adaptativas.

1.2.1 La política: estrategia de acción

La **política** representa la estrategia que sigue un agente para elegir una acción en cada estado. Puede ser determinista, si asigna una acción concreta a cada estado, o estocástica, si define una distribución de probabilidades sobre posibles acciones.

En ambos casos, es el componente central que el agente ajusta durante su aprendizaje para maximizar las recompensas acumuladas a lo largo del tiempo.

Ejemplos:

En un sistema de recomendación como el de Amazon, la política podría consistir en seleccionar el producto con mayor probabilidad de ser adquirido, según el historial del usuario. Esta elección se refina conforme el sistema recoge nuevos datos de interacción, permitiendo una adaptación progresiva.

Un aspecto relevante es cómo se parametriza la política. En los métodos tabulares, las políticas se representan como matrices explícitas, útiles en espacios pequeños. En cambio, en entornos complejos, se utilizan funciones aproximadoras como redes neuronales, que permiten generalizar a estados no vistos.

Algoritmos como **SARSA o Q-learning** utilizan distintas estrategias para actualizar sus políticas. SARSA ajusta la política teniendo en cuenta la acción que el agente realmente elige, mientras que Q-learning lo hace considerando la mejor acción posible en el siguiente estado, independientemente de la que realmente se seleccione. Esta diferencia tiene implicaciones sobre la exploración y la estabilidad del aprendizaje (Gavilán, s.f.).

1.2.2 Tipos de políticas y sus aplicaciones

Las políticas pueden dividirse en **exploratorias, explotadoras o balanceadas**. Las primeras se orientan a descubrir nuevas estrategias, mientras que las explotadoras se apoyan en el conocimiento ya adquirido. Las balanceadas intentan combinar lo mejor de ambas.

Figura 4: Tipos de políticas

En entornos de e-commerce, una política exploratoria podría presentar productos nuevos o poco populares para detectar intereses ocultos.

Por su parte, una política explotadora repetirá las recomendaciones que han tenido éxito con perfiles similares.

Muchos sistemas reales implementan mecanismos como decaimiento progresivo de la exploración para adaptarse a la fase del ciclo de vida del producto.

1.2.3 La función de valor: evaluación del estado

La **función de valor**, denotada habitualmente como $V(s)$, mide la bondad de estar en un determinado estado s, suponiendo que el agente seguirá su política actual. Se calcula como el valor esperado de la suma de recompensas futuras descontadas. Este descuento se introduce mediante un parámetro γ, que determina la importancia relativa de recompensas futuras frente a inmediatas.

Esta función permite al agente saber **cuáles estados conducen**, en promedio, a **mayores beneficios**. En contextos empresariales como la gestión de almacenes, un estado con niveles de inventario equilibrados y alta disponibilidad podría tener un alto valor, mientras que estados con roturas de stock o exceso de existencias recibirían valores bajos.

El cálculo de $V(s)$ puede hacerse por estimación directa, simulando trayectorias desde s, o mediante aprendizaje temporal (temporal difference learning), ajustando las estimaciones a medida que se recogen nuevas experiencias.

Este enfoque permite la actualización continua del conocimiento del agente sin esperar a que finalice cada episodio.

Ejemplo:

En servicios financieros, un sistema que gestiona portafolios puede utilizar $V(s)$ para evaluar combinaciones de activos. Estados con diversificación adecuada y bajo riesgo sistémico obtendrán valores superiores.

1.2.4 Función de valor con métodos aproximados

Cuando el espacio de estados es demasiado grande para utilizar tablas, se emplean funciones de aproximación como regresores lineales o redes neuronales profundas. Estas herramientas permiten extrapolar valores para estados nuevos, generalizando el aprendizaje y aumentando la escalabilidad del sistema.

El uso de técnicas como redes convolucionales en visión por computador ha permitido evaluar estados visuales en robótica de forma eficiente, proporcionando a $V(s)$ un papel clave en la navegación de entornos desconocidos.

1.2.5 La función Q: valor de la acción

La **función Q**, o función de valor de acción, se representa como $Q(s, a)$ y mide la utilidad de tomar la acción a en el estado s, suponiendo que el agente seguirá su política actual a partir de ese punto. A diferencia de la función de valor, que solo considera estados, la función Q permite comparar directamente diferentes acciones en un mismo contexto.

Ejemplos:

En un sistema de fijación de precios como el de Ryanair, $Q(s, a)$ podría estimar el beneficio esperado de asignar cierto precio a a un billete en una fecha determinada s. El agente puede evaluar opciones como subir, bajar o mantener el precio, y elegir la que maximice el retorno esperado.

El uso de $Q(s, a)$ es la base de algoritmos como Q-learning, donde el agente actualiza sus valores Q utilizando la fórmula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Este método permite aprender una política óptima incluso sin conocer previamente el modelo del entorno. Las redes neuronales permiten aproximar $Q(s, a)$ en problemas donde el número de pares estado-acción es enorme.

1.2.6 Aprendizaje profundo aplicado a la función Q

El uso de redes neuronales profundas ha transformado la implementación de $Q(s, a)$.

Deep Q-Networks (DQN) entrenan un modelo que predice el valor de todas las acciones posibles para una entrada dada, permitiendo decisiones eficientes en tiempo real.

Ejemplos:

En entornos como la conducción autónoma, esta técnica permite al sistema elegir entre frenar, acelerar o girar según el contexto visual, obteniendo estimaciones precisas del resultado esperado de cada acción.

1.2.7 Comparativa pedagógica

A continuación, se presenta una tabla comparativa que permite visualizar de forma clara las diferencias funcionales y operativas entre estos tres elementos, facilitando su asimilación conceptual y su aplicación práctica en entornos reales.

Componente	Descripción	**Entrada**	**Salida**	**Uso principal**
Política $\pi(s)$	Decide qué acción ejecutar	Estado s	Acción a	Tomar decisiones
Valor $V(s)$	Evalúa la calidad de un estado	Estado s	Valor esperado	Priorizar estados
Valor-Acción $Q(s, a)$	Evalúa la calidad de una acción en un estado	Estado y acción s, a	Valor esperado	Comparar acciones

Tabla 2: Comparación entre política, función de valor y función Q.

¿Sabías que?

El algoritmo Deep Q-Network (DQN), desarrollado por DeepMind, combina la función Q con redes neuronales profundas para aprender directamente de entradas visuales en videojuegos. Este hito demostró que un agente podía superar a humanos en juegos clásicos de Atari, aprendiendo desde cero solo con la información de la pantalla y las recompensas (Dávila, s.f.).

1.2.8 Exploración vs explotación en políticas

Las políticas estocásticas permiten que el agente explore nuevas acciones que podrían ser mejores a largo plazo.

Este proceso se equilibra con la **explotación** de las acciones conocidas que ofrecen buenos resultados. Una técnica habitual es ϵ-greedy, donde con probabilidad ϵ se elige una acción aleatoria y con probabilidad $1-\epsilon$ se elige la mejor según la función Q.

Otras empresas emplean estrategias de **exploración**, por ejemplo, controlada en sus sistemas de recomendación para descubrir nuevos contenidos relevantes, evitando caer en bucles de retroalimentación donde los usuarios solo reciben más de lo mismo.

1.2.9 Aplicaciones prácticas en entornos empresariales

La elección de la política y el cálculo de las funciones de valor tienen impacto directo en aplicaciones como el mantenimiento predictivo, la optimización de rutas o la detección de fraude. En estos contextos, disponer

de una estimación precisa del valor esperado de las acciones permite anticiparse a problemas y actuar con eficiencia.

Ejemplos:

En bancos como **BBVA**, se han explorado sistemas que utilizan Q-learning para decidir cuándo y a quién ofrecer ciertos productos financieros, maximizando la aceptación y reduciendo el riesgo de sobreoferta.

Empresas logísticas como **FedEx** integran funciones Q en sus modelos de optimización de rutas para reducir el tiempo de entrega sin aumentar los recursos empleados, contribuyendo a una mayor eficiencia operativa.

¿Sabías que?

En el entorno de las criptomonedas, algunos agentes de trading utilizan variantes del aprendizaje por refuerzo para decidir operaciones de compra y venta. Estas estrategias, si bien aún experimentales, han mostrado resultados prometedores en mercados altamente volátiles y sensibles a pequeños cambios.

Tema 2. Algoritmos comunes

El aprendizaje por refuerzo se apoya en algoritmos que permiten al agente mejorar sus decisiones a partir de la experiencia acumulada en el entorno.

Estos algoritmos establecen la manera en que se actualizan las políticas de acción y las funciones de valor, con el fin de maximizar las recompensas obtenidas a largo plazo. Su conocimiento permite entender cómo los sistemas aprenden estrategias eficaces sin necesidad de supervisión directa ni ejemplos etiquetados.

Comprender el funcionamiento de métodos como Q-learning o SARSA permite no solo aplicar técnicas concretas, sino también interpretar los resultados que generan, evaluar su desempeño y adaptar su uso a distintas situaciones prácticas.

Estos enfoques aportan herramientas versátiles que han demostrado ser eficaces en aplicaciones tan diversas como la robótica, la optimización de recursos o el desarrollo de agentes inteligentes en videojuegos.

2.1 Q-learning

El algoritmo **Q-learning** se sitúa como uno de los más relevantes y ampliamente utilizados dentro del aprendizaje por refuerzo. Su capacidad para aprender políticas óptimas sin requerir un modelo del entorno lo convierte en una herramienta especialmente versátil y adecuada para numerosos contextos prácticos.

La base de Q-learning reside en la estimación de la función de valor de acción, conocida como **función Q**, que permite al agente tomar decisiones informadas sobre qué acción ejecutar en cada estado.

2.1.1 Fundamentos del algoritmo Q-learning

El algoritmo Q-learning tiene como objetivo aprender la función $Q(s, a)$, que representa el valor esperado de la recompensa acumulada que se obtendrá tomando la acción a en el estado s, y siguiendo una política óptima a partir de ese punto.

A diferencia de otros métodos como SARSA, Q-learning es un método fuera de la política (*off-policy*), lo que significa que **aprende el valor de la mejor acción posible,** independientemente de la que realmente se haya ejecutado durante la experiencia.

La actualización de la función Q se realiza según la siguiente fórmula:

$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$

Donde:

- **α** es la tasa de aprendizaje, que determina el peso de la nueva información respecto a la antigua.
- r es la recompensa obtenida tras realizar la acción a en el estado s.
- **γ** es el factor de descuento, que controla la importancia de las recompensas futuras.
- **s'** es el nuevo estado alcanzado tras la acción.
- **$\max_{a'} Q(s', a')$** es el valor estimado de la mejor acción posible en el nuevo estado.

2.1.2 Aplicaciones en entornos empresariales

Q-learning ha sido aplicado con éxito en múltiples sectores empresariales para resolver problemas de decisión secuencial.

En el **ámbito logístico**, por ejemplo, se utiliza para optimizar rutas de entrega, considerando variables como tráfico, disponibilidad de vehículos y prioridad de pedidos. DHL y UPS han explorado variantes de Q-learning para reducir los tiempos de entrega y los costes operativos.

En **marketing digital**, empresas como Booking.com han investigado su uso para decidir qué ofertas mostrar a cada usuario, adaptando la estrategia en función del historial de interacciones y aumentando la conversión. La

flexibilidad del algoritmo permite incorporar nuevas fuentes de información sin rediseñar por completo el modelo.

2.1.3 Estructura de la tabla Q y su interpretación

En su versión tabular, Q-learning mantiene una tabla donde cada combinación de estado y acción tiene asociado un valor que refleja su utilidad estimada. Esta tabla se actualiza de forma iterativa a medida que el agente interactúa con el entorno.

Estado s	Acción a_1	Acción a_2	Acción a_3
s_1	4.5	3.2	2.1
s_2	1.0	5.6	4.0

Tabla 3: Ejemplo de tabla Q simplificada para un entorno discreto.

Cada celda contiene el valor Q que orienta al agente hacia la acción más prometedora. A medida que el agente obtiene experiencia, estos valores convergen hacia la estimación real del retorno esperado.

2.1.4 Exploración y explotación: el dilema clásico

Un aspecto determinante en el aprendizaje mediante Q-learning es el equilibrio entre **exploración** y **explotación**.

Explorar permite al agente descubrir nuevas estrategias, mientras que **explotar** implica utilizar las mejores acciones conocidas hasta el momento. El método ϵ-greedy es una técnica común que introduce aleatoriedad controlada en la selección de acciones.

Ejemplos:

Un sistema que recomienda contenido en una plataforma educativa como Coursera puede beneficiarse de presentar ocasionalmente cursos nuevos que el usuario no ha explorado, en lugar de repetir únicamente aquellos que tienen mayor probabilidad de éxito.

2.1.5 Ventajas y limitaciones del algoritmo

Entre las principales **ventajas** de Q-learning se encuentra su capacidad para aprender sin modelo del entorno, lo que lo hace apropiado para escenarios donde no se conocen a priori las consecuencias de cada acción. Además, su implementación es relativamente sencilla y escalable en entornos discretos.

Sin embargo, sus **limitaciones** inciden en su rendimiento que disminuye cuando el espacio de estados y acciones se vuelve muy amplio o continuo.

En estos casos, las versiones tabulares se vuelven impracticables, y es necesario recurrir a **funciones de aproximación**, como redes neuronales, para representar $Q(s, a)$. Este salto conduce a técnicas como Deep Q-Networks, que amplían el alcance del algoritmo a entornos complejos y de alta dimensionalidad.

¿Sabías que?

El juego Ms. Pac-Man fue uno de los primeros entornos donde se demostró la eficacia de Q-learning con funciones de aproximación. El agente aprendía estrategias evasivas y ofensivas sin instrucciones explícitas, mejorando sus resultados tras miles de iteraciones.

2.1.6 Integración con redes neuronales: hacia DQN

La integración de Q-learning con redes neuronales profundas dio lugar al desarrollo de los algoritmos **Deep Q-Networks (DQN)**. Estos modelos permiten representar funciones Q en entornos con representaciones complejas, como imágenes o secuencias temporales.

DeepMind aplicó DQN para superar el rendimiento humano en varios videojuegos de Atari, utilizando como entrada exclusivamente la imagen en bruto de la pantalla. El modelo aprendía de la experiencia almacenada en una memoria de repetición, lo que permitía decorrelacionar las muestras y estabilizar el aprendizaje.

2.1.7 Casos prácticos destacados

En la **industria manufacturera**, General Electric ha experimentado con Q-learning para ajustar parámetros de máquinas de forma dinámica, con el objetivo de minimizar defectos y optimizar la calidad del producto.

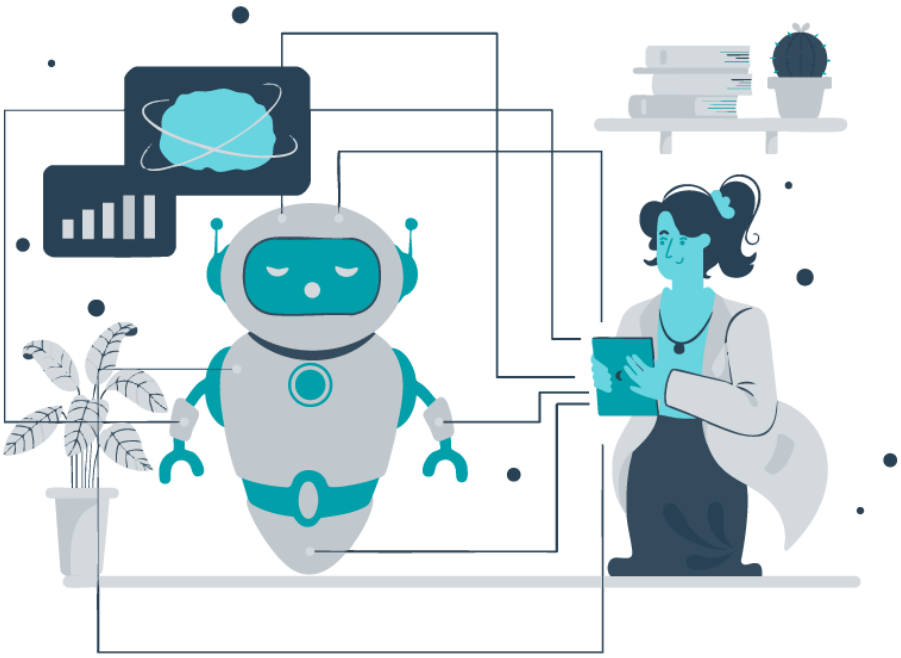
En el **sector de la energía**, algunas compañías han empleado Q-learning para equilibrar la generación y el consumo en redes inteligentes. El agente decide en tiempo real cuándo activar fuentes renovables o almacenar

energía, maximizando la eficiencia sin comprometer la estabilidad del sistema.

En **e-commerce**, plataformas como eBay han integrado componentes de aprendizaje por refuerzo en su estrategia de fijación de precios y subastas, ajustando los parámetros en función del comportamiento del mercado y la elasticidad de la demanda.

2.2 SARSA

El algoritmo **SARSA** es una de las variantes más relevantes dentro del aprendizaje por refuerzo. Su denominación proviene de la secuencia **Estado-Acción-Recompensa-Estado-Acción**, que define su dinámica de actualización.



A diferencia de Q-learning, que es un método fuera de la política (*off-policy*), SARSA es un método **dentro de la política** (*on-policy*), lo que significa que **aprende a partir de las acciones que realmente ejecuta el agente**, lo que tiene implicaciones sobre la estabilidad, robustez y adaptabilidad del sistema.

2.2.1 Fundamentos del algoritmo SARSA

SARSA tiene como objetivo aprender la función de valor de acción $Q(s, a)$, es decir, la utilidad esperada de tomar una acción a en un estado s y continuar con la política actual. Su fórmula de actualización es:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a)\right]$$

Donde:

- **s**: estado actual.
- **a**: acción elegida en s.
- **r**: recompensa recibida tras ejecutar a.
- **s'**: nuevo estado alcanzado.
- **a'**: acción elegida en s'.
- **α**: tasa de aprendizaje.
- **γ**: factor de descuento de recompensas futuras.

El aprendizaje se basa en la **experiencia real del agente**, utilizando la acción a' que efectivamente se selecciona en el siguiente estado. Esta estrategia tiende a ser más conservadora que Q-learning, lo que puede resultar beneficioso en entornos inciertos o ruidosos.

2.2.2 Diferencias clave entre SARSA y Q-learning

Aunque SARSA y Q-learning comparten la estructura general de aprendizaje por refuerzo, sus diferencias metodológicas determinan resultados distintos en ciertos entornos.

Mientras Q-learning actualiza sus estimaciones usando la mejor acción posible en el nuevo estado (independientemente de la que realmente se elija), SARSA ajusta los valores Q según la acción efectivamente seleccionada.

Esto hace que **SARSA tienda a ser más cauto y menos agresivo**, lo que puede evitar estrategias demasiado optimistas en entornos con transiciones estocásticas. En contextos de negocios donde existen penalizaciones significativas por decisiones arriesgadas, SARSA se adapta mejor a la realidad operacional.

En la siguiente tabla se expone una comparación entre SARSA y Q-learning.

Característica	SARSA	Q-learning
Tipo de aprendizaje	Dentro de la política	Fuera de la política
Valor futuro utilizado	$Q(s', a')$	$\max_{a'} Q(s', a')$
Naturaleza	Prudente	Optimista
Adecuado para	Entornos estocásticos	Entornos deterministas

Tabla 4: Diferencias operativas entre los algoritmos SARSA y Q-learning.

2.2.3 Aplicaciones del algoritmo SARSA en la empresa

SARSA ha sido utilizado en diversos sectores para entrenar sistemas que requieren precisión y adaptabilidad.

En el sector de la **automoción**, algunas marcas han implementado SARSA en el desarrollo de vehículos autónomos, especialmente en maniobras de baja velocidad donde el riesgo es alto y la reacción del entorno poco predecible.

En **logística**, se ha aplicado en la gestión de almacenes para el control de robots autónomos que trasladan productos. A diferencia de Q-learning, SARSA reduce la probabilidad de que los robots realicen movimientos peligrosos o ineficientes, porque aprende de su propia trayectoria real y no de estimaciones teóricas.

2.2.4 Políticas de exploración en SARSA

El aprendizaje efectivo en SARSA depende de una combinación adecuada entre exploración y explotación.

El estilo ϵ-greedy permite mantener la variabilidad necesaria en la selección de acciones, especialmente en las fases iniciales del entrenamiento. Con el tiempo, ϵ se reduce para favorecer la explotación del conocimiento adquirido.

Ejemplos:

En un entorno de atención al cliente automatizada, por ejemplo, el uso de SARSA con una política ϵ-greedy puede permitir a un bot aprender cuándo transferir una conversación a un agente humano. Al principio probará distintas opciones, pero con la experiencia afinará su criterio basándose en resultados anteriores.

2.2.5 Integración con funciones de aproximación

Al igual que otros algoritmos tabulares, SARSA enfrenta limitaciones en espacios de estados o acciones continuos. Para superarlas, se pueden utilizar **aproximadores de funciones**, como redes neuronales o métodos basados en árboles de decisión.

Esta extensión ha permitido utilizar SARSA en domínios como la previsión de demanda, donde los estados incluyen secuencias de ventas, precios, promociones y variables externas como clima o eventos. El modelo estima $Q(s, a)$ con suficiente generalización como para ser aplicado en situaciones no vistas previamente.

¿Sabías que?

SARSA fue uno de los algoritmos utilizados en simuladores de vuelo para entrenar a sistemas de asistencia inteligente. Estos sistemas debían actuar con prudencia y adaptarse a condiciones variables sin asumir riesgos innecesarios, lo que lo convertía en un candidato idóneo frente a otros modelos más agresivos como Q-learning.

2.2.6 Evaluación de rendimiento y convergencia

La evaluación del rendimiento de SARSA se realiza mediante la comparación entre las recompensas acumuladas y los valores esperados de $Q(s, a)$. Es habitual representar gráficamente la evolución de la recompensa media por episodio y analizar su estabilización. La convergencia ocurre cuando los valores Q dejan de cambiar significativamente entre episodios.

Ejemplos:

En experimentos de control de tráfico urbano, SARSA ha demostrado mayor estabilidad que Q-learning cuando las condiciones cambian rápidamente (por ejemplo, accidentes o desvíos), ya que su estilo de aprendizaje está más alineado con la experiencia real.

2.2.7 SARSA(λ): extensión con trazas de elegibilidad

Una extensión de SARSA es **SARSA(λ)**, que incorpora un mecanismo de **trazas de elegibilidad**. Este mecanismo permite actualizar no solo la acción reciente, sino también aquellas que contribuyeron al estado actual, ponderadas por su antigüedad. Esta técnica acelera el aprendizaje al distribuir crédito más allá de una única transición.

Ejemplos:

En el ámbito de la automatización de procesos de negocio (BPA), SARSA (λ) ha sido considerado para entrenar bots que optimizan flujos de tareas. Al incluir una memoria de acciones pasadas, el sistema identifica patrones eficientes que no son inmediatamente visibles con SARSA básico.

2.2.8 Casos de uso sectoriales

En **sanidad**, centros hospitalarios han utilizado SARSA para asignar recursos críticos, como camas o quirófanos, de forma dinámica según la demanda cambiante. La naturaleza estocástica de los ingresos y urgencias hace que SARSA sea preferido frente a métodos más agresivos.

En **fintech**, startups que ofrecen microcréditos han evaluado SARSA para ajustar límites de crédito de manera personalizada. El algoritmo aprende cuándo es adecuado ampliar el límite en base al comportamiento reciente del usuario, minimizando el riesgo sin bloquear oportunidades de negocio.

En la **industria del entretenimiento**, plataformas de streaming han implementado SARSA en la gestión de recomendaciones contextuales durante eventos en directo, adaptando la experiencia sin interrumpir al espectador.

2.3 Aplicaciones reales (juegos, robótica)

El aprendizaje por refuerzo ha dejado de ser una disciplina teórica para convertirse en una tecnología aplicada de forma directa en diversos contextos reales.

Este apartado profundiza en dos de los campos donde sus resultados han sido más visibles y determinantes: el desarrollo de videojuegos y la

robótica. Ambos dominios presentan entornos dinámicos, inciertos y con múltiples variables interconectadas, lo que los convierte en escenarios ideales para poner a prueba algoritmos de toma de decisiones secuenciales.

A lo largo de esta sección se analizarán casos concretos, arquitecturas implementadas y ventajas obtenidas al integrar modelos de aprendizaje por refuerzo en estas áreas, así como también se reflexionará sobre sus limitaciones y desafíos actuales.

Además, se destacarán avances recientes en sectores híbridos como los vehículos autónomos o la agricultura robotizada, que combinan características de ambos mundos.

2.3.1 Introducción al impacto aplicado del aprendizaje por refuerzo

El **aprendizaje por refuerzo (RL)** ha trascendido los laboratorios académicos y se ha incorporado progresivamente en aplicaciones del mundo real.

Desde sistemas de navegación autónoma hasta plataformas de entretenimiento interactivo, su implementación demuestra un potencial estratégico decisivo para mejorar decisiones secuenciales, adaptarse a entornos cambiantes y aprender de forma autónoma en escenarios complejos.

Dos de los ámbitos más destacados para aplicar estos algoritmos son el desarrollo de videojuegos y los sistemas robóticos, donde la interacción dinámica con el entorno resulta ideal para explotar las capacidades de RL.

2.3.2 Aplicaciones en videojuegos: del entrenamiento a la autonomía

El sector de los **videojuegos** ha sido históricamente un campo fértil para experimentar con técnicas de inteligencia artificial. Los algoritmos de RL han sido integrados en distintos géneros, desde juegos de estrategia hasta simuladores deportivos, con el propósito de optimizar el comportamiento de los agentes virtuales.

Ejemplos:

Un ejemplo paradigmático lo constituye el caso de DeepMind y su aplicación AlphaGo, que logró vencer a campeones mundiales del juego de mesa Go gracias a técnicas avanzadas de aprendizaje por refuerzo profundo (Dávila, 2022).

Otro ejemplo relevante es el trabajo de OpenAI Five en el entorno competitivo del videojuego Dota 2, donde un equipo de agentes entrenados por RL fue capaz de cooperar y tomar decisiones tácticas superiores a las de jugadores humanos profesionales.

Este proyecto no sólo destacó por su rendimiento, sino también por su capacidad para generar estrategias no previstas por los programadores humanos.

¿Sabías que?

En el entorno del videojuego StarCraft II, DeepMind desarrolló AlphaStar, un agente que llegó a posicionarse entre los mejores jugadores del mundo, siendo uno de los primeros sistemas de RL en competir contra humanos en un entorno en tiempo real con información parcial.

2.3.3 Arquitecturas y metodologías empleadas en videojuegos

Para lograr estos avances, las arquitecturas empleadas suelen combinar técnicas de **Q-learning profundo (Deep Q-Network, DQN)** con exploración por políticas y mecanismos de experiencia acumulada.

Los entornos se simulan con rapidez y permiten ejecutar millones de iteraciones, ajustando los pesos de redes neuronales que aprenden a maximizar la recompensa esperada mediante la interacción reiterada con el entorno del juego.

Además de la optimización de agentes no jugadores (NPC), el RL se ha empleado en la **generación procedural de contenidos** y en la **adaptación dinámica de la dificultad** según el perfil del usuario, mejorando la experiencia de juego y manteniendo el interés de los jugadores.

2.3.4 Aplicaciones robóticas: autonomía y aprendizaje adaptativo

En el ámbito de la robótica, el aprendizaje por refuerzo ha mostrado una gran capacidad para entrenar robots que ejecutan tareas complejas con precisión creciente. Desde brazos robóticos que manipulan objetos hasta drones que navegan de forma autónoma, los agentes RL están siendo

entrenados para percibir, decidir y actuar con niveles elevados de autonomía.

Ejemplo:

En el caso de **Boston Dynamics,** los modelos de RL permiten que robots como Spot realicen caminatas adaptativas en terrenos irregulares, mejorando su equilibrio y resistencia. De igual forma, Amazon Robotics ha incorporado RL en su sistema de optimización de rutas para robots logísticos, reduciendo el tiempo de recogida de productos y maximizando la eficiencia energética en sus centros de distribución.

2.3.5 Casos reales en entornos industriales

En el sector industrial, la implementación del RL se está utilizando en robots para inspección visual y mantenimiento predictivo.

Compañías como **Siemens** están desarrollando sistemas que emplean aprendizaje por refuerzo para optimizar líneas de producción y mejorar el ensamblaje automatizado mediante estrategias que ajustan parámetros de control en tiempo real.

¿Sabías que?

En algunos laboratorios de investigación avanzada, se ha logrado entrenar brazos robóticos para resolver cubos Rubik de forma autónoma, sin intervención humana directa, solo a partir de experiencia simulada y recompensa diferida.

Una aplicación destacada del RL en robótica es la **asistencia médica personalizada**. Robots entrenados con aprendizaje por refuerzo pueden colaborar en tareas como la entrega de medicación, el acompañamiento de pacientes con movilidad reducida o incluso realizar intervenciones quirúrgicas mínimamente invasivas bajo supervisión. Un ejemplo lo ofrece el robot Da Vinci, mejorado con RL para ajustar con precisión los movimientos del instrumental quirúrgico.

También se han desarrollado robots sociales como **Pepper o Nao**, cuyo comportamiento puede adaptarse al entorno hospitalario o educativo mediante técnicas de RL que ajustan el tono de voz, distancia interpersonal y gestos, con el objetivo de generar empatía y mejorar la interacción con los usuarios.

Actualmente se está explorando el uso de RL en **vehículos autónomos**, tanto terrestres como aéreos, donde los agentes deben tomar decisiones en entornos complejos, dinámicos y altamente inciertos. Empresas como Waymo y Tesla están incorporando principios de RL en la planificación de rutas, detección de obstáculos y ajuste de velocidades.

Otro sector en auge es el de la **agricultura robotizada**, donde tractores autónomos, drones fumigadores y recolectores aprenden a optimizar rutas, identificar plagas o seleccionar frutos maduros mediante RL combinado con visión por computador.

2.3.6 Comparativa entre entornos virtuales y físicos

Una distinción relevante radica en la diferencia entre entrenar un agente en un entorno simulado y uno físico. En el primer caso, los algoritmos pueden realizar millones de iteraciones rápidamente, permitiendo ajustes veloces de políticas y estrategias.

En cambio, los entornos físicos imponen restricciones técnicas como el desgaste de materiales, tiempos de ejecución prolongados y riesgos asociados a errores. Para mitigar esto, muchos proyectos utilizan **entornos simulados con posterior transferencia al mundo real**, un proceso conocido como **sim-to-real**.

En la siguiente tabla se realiza una comparativa entre aplicaciones en videojuegos y robótica.

Característica	Videojuegos	Robótica física
Tipo de entorno	Simulado, virtual	Físico, real
Velocidad de entrenamiento	Muy rápida	Limitada por restricciones físicas
Riesgo por errores del agente	Nulo	Alto (colisiones, daños)
Flexibilidad de tareas	Alta (programable)	Media (limitada por hardware)
Transferencia al mundo real	No necesaria	Obligatoria en la mayoría de aplicaciones

Tabla 5: Comparación entre aplicaciones en videojuegos y robótica

Resumen

El Módulo 4 introduce al estudiante en los principios del aprendizaje por refuerzo, una técnica en la que un agente aprende a tomar decisiones mediante la interacción con un entorno y la retroalimentación en forma de recompensas.

Se analizan los elementos clave del sistema: el agente, que toma decisiones; el entorno, que responde a las acciones; y la recompensa, que guía el aprendizaje.

También se estudian la política, la función de valor y la función Q, esenciales para que el agente determine qué acción tomar. Se presentan dos algoritmos relevantes: Q-learning, eficaz en entornos conocidos, y SARSA, más prudente en situaciones inciertas.

Por último, se destacan aplicaciones prácticas en videojuegos y robótica, donde estos métodos permiten desarrollar comportamientos adaptativos y autónomos. ¡Esperamos que este resumen os haya sido útil!

www.awakelab.worl

MÓDULO 4:

Aprendizaje por Refuerzo

AWAKELAB | Desarrollado por Awakelab.world